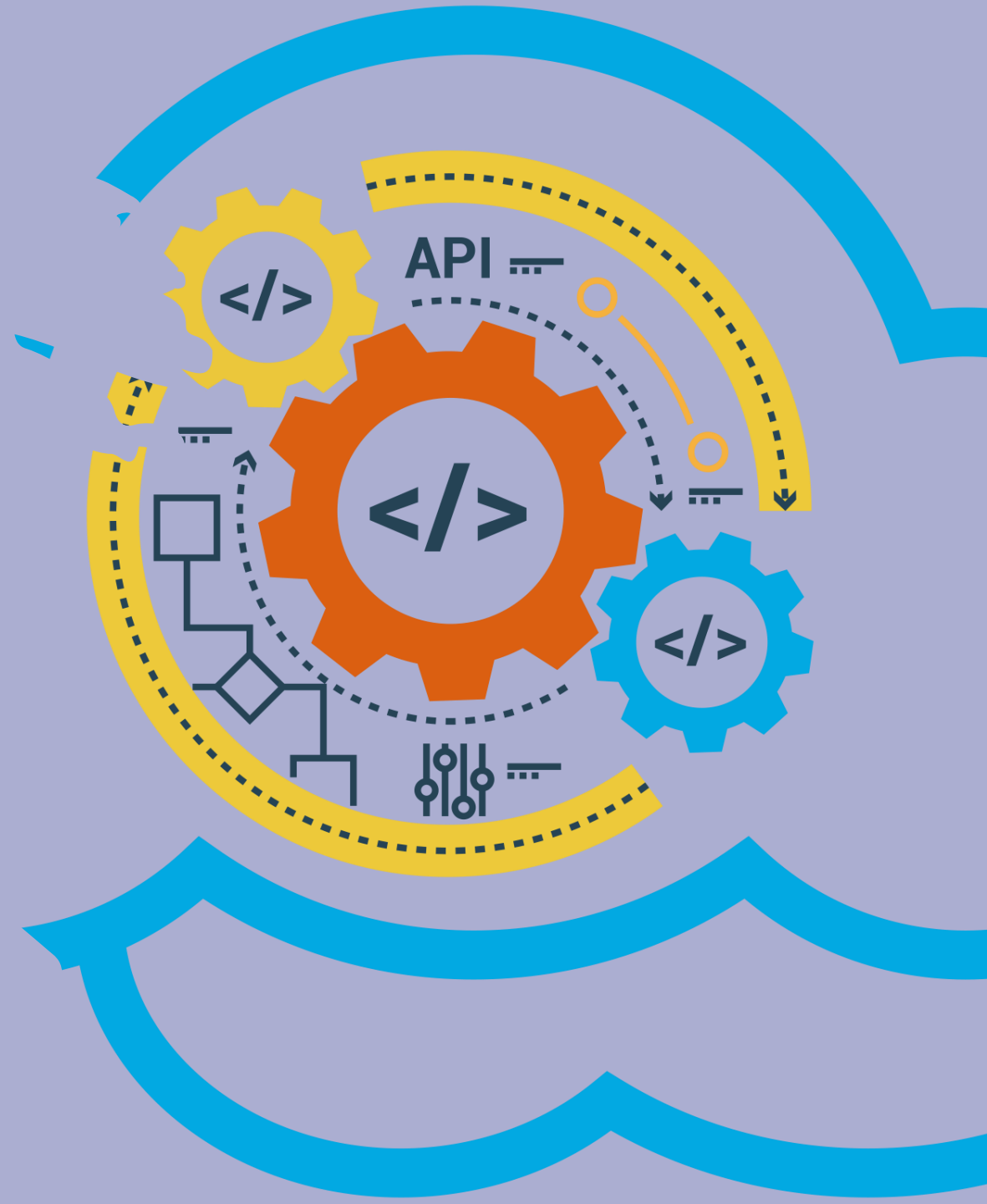
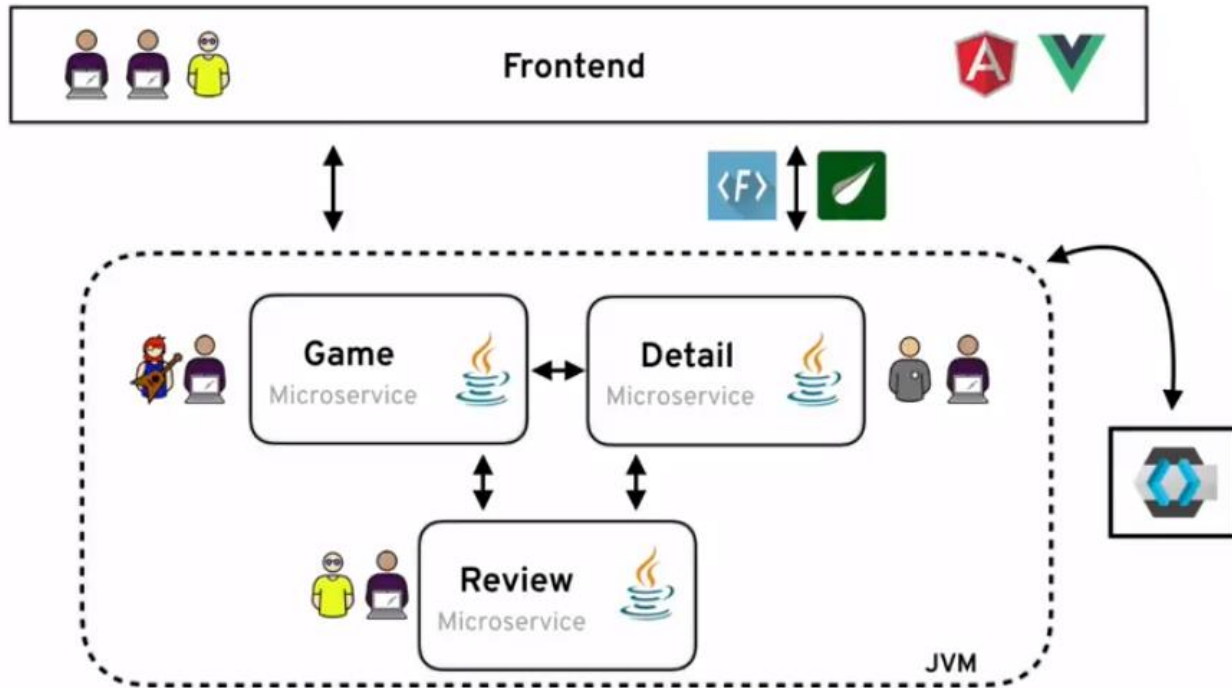


Microservicios

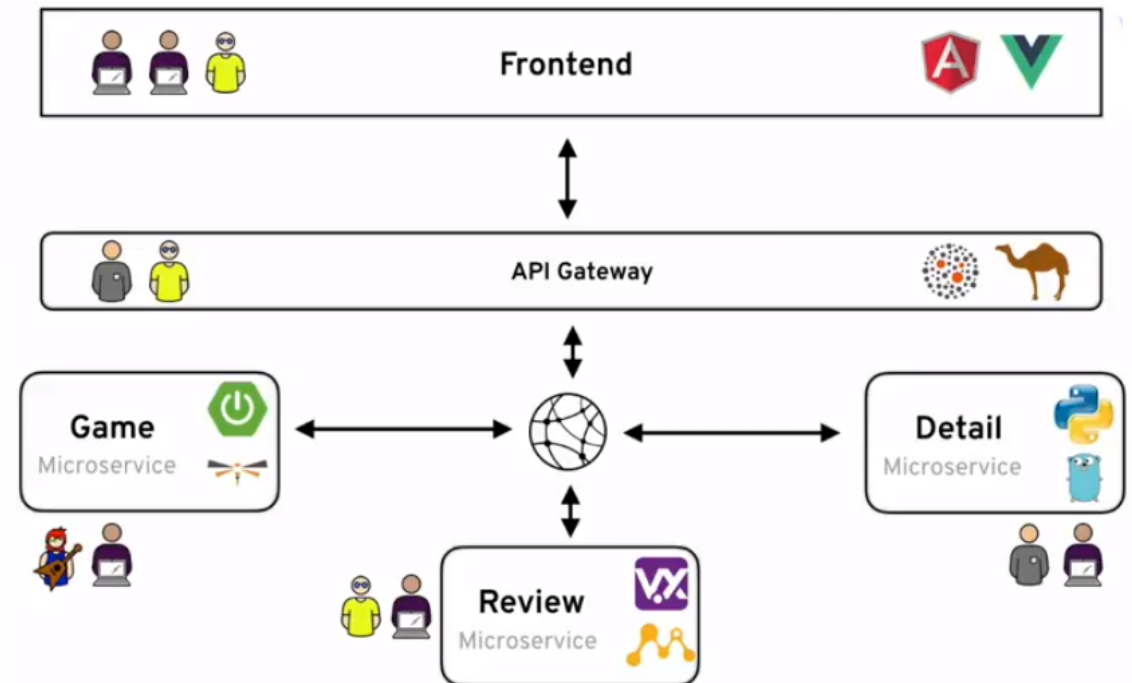
Instructor: Ilse Aline Morales
Castillo



Arquitectura monolitica



Arquitectura de Microservicios



Arquitectura de microservicios

- La arquitectura de microservicios es un enfoque de diseño de software donde una aplicación se construye como un conjunto de servicios pequeños e independientes, cada uno ejecutando un proceso único y enfocado en una tarea específica. Estos servicios son desarrollados, implementados y escalados de forma independiente, lo que permite a los equipos de desarrollo trabajar de manera más ágil y eficiente.

Características



Alto nivel de desacoplamiento: cada microservicio tiene su propio dominio, comunicándose y compartiendo datos con los demás



Nombres únicos: cada microservicio tiene su propio nombre para así poder decidir su ubicación



Resiliencia: En caso de que den algún problema, pueden iniciarse en otra máquina de forma que así no perderemos ningún dato ni información



Independencia: su implementación, escalado y actualización se hace de manera independiente



Escenario profesional: están enfocados para desarrollarse en función de cada negocio o de cada cliente por lo tanto son más sencillos y están mucho menos condicionados por la tecnología



ESCALABILIDAD



MÍNIMO EQUIPO DE TRABAJO



MÓDULOS INDEPENDIENTES



FUNCIONALIDAD MODULAR



TOTAL, LIBERTAD DEL
DESARROLLADOR A LA HORA
DE DESARROLLAR Y
DESPLEGAR SERVICIOS DE
MANERA INDEPENDIENTE



AL UTILIZAR CONTENEDORES,
PERMITE EL DESPLIEGUE Y EL
DESARROLLO MÁS RÁPIDO Y
EFICIENTE DE LA APLICACIÓN.

Ventajas

Desventajas

Alto consumo de memoria

Necesidad de tiempo para poder fragmentar distintos microservicios

Complejidad de gestión de un gran número de servicios

Necesidad de desarrolladores para la solución de problemas como latencia en la red o balanceo de cargas

Pruebas o tests complicados al despliegue distribuido

¿Por qué usar Microservicios?



Habilita la entrega continua



Permite un flujo de trabajo de desarrollador eficiente porque son altamente fáciles de mantener



Son robustos por diseño



Puede escalar hacia arriba o hacia abajo independientemente uno del otro

Ejemplos:

Un buen ejemplo del uso de la arquitectura de microservicios es Walmart. La arquitectura se implementó en 2012 tras constantes problemas durante los picos de actividad de los clientes. El flujo de trabajo era imperfecto y, debido a los problemas del sistema, se produjo un notable descenso de los ingresos.

Después de auditar el sistema existente, la empresa decidió cambiar a microservicios. Con la ayuda de este enfoque, fue posible redistribuir las tareas y simplificar así el flujo de trabajo.

He aquí algunos resultados tras la transición:

- El consumo de energía del ordenador se redujo al 40%.
- Gracias al cambio de arquitectura, el gasto en hardware se redujo a un mínimo del 20% y a un máximo del 50% en algunos casos.
- Se hizo posible lograr un uso completo de las aplicaciones móviles. Así, los microservicios aumentaron la cantidad de conversiones a través de otro flujo de compras.
- Se evitó el tiempo de inactividad durante las rebajas de Navidad, el Black Friday y otros picos de actividad de los clientes.



Buenas Practicas



1.- Descomposición adecuada: Dividir la funcionalidad en microservicios cohesivos, evitando tamaños excesivamente pequeños o grandes.



2.- Independencia de implementación: Cada microservicio debe ser autónomo, con su propia base de código, datos y despliegue, minimizando dependencias.



3.- Límites de contexto claros: Definir responsabilidades y datos de cada microservicio para evitar duplicación de funcionalidad.



4.- Comunicación asincrónica: Utilizar mensajes o eventos asincrónicos en lugar de comunicación síncrona para reducir el acoplamiento.



5.- Gestión de errores y resiliencia: Implementar mecanismos para manejar errores y fallos de forma resiliente.



6.- Monitorización y trazabilidad: Establecer prácticas para seguir el rendimiento y estado de los microservicios, identificando posibles problemas.



7.- Automatización y CI/CD: Automatizar procesos de construcción, prueba, despliegue y escala para una entrega continua rápida y segura.



8.- Seguridad: Implementar medidas de seguridad como autenticación, autorización y protección contra vulnerabilidades.



9.- Documentación y versionado: Mantener documentación actualizada de cada microservicio, gestionando el versionado para facilitar la actualización y compatibilidad.



10.- Pruebas exhaustivas: Realizar pruebas unitarias, de integración y de extremo a extremo para garantizar el correcto funcionamiento y compatibilidad entre componentes.

¿Qué es eureka server?

Es un componente central en la arquitectura de microservicios proporcionado por Netflix como parte del conjunto de herramientas de Netflix OSS y utilizado comúnmente en el ecosistema Spring Cloud. Es un servicio de descubrimiento que permite a los microservicios registrarse y descubrir otros servicios sin necesidad de configurar manualmente sus ubicaciones



Funcionalidades de Eureka Server

Registro de Servicios: Los microservicios se registran en el Eureka Server, proporcionando su información de red (como dirección IP y puerto). Esta información se almacena en un registro de servicios centralizado.

Descubrimiento de Servicios: Los microservicios pueden consultar el Eureka Server para encontrar la ubicación de otros servicios registrados. Esto permite a los servicios comunicarse con otros servicios de manera dinámica, sin depender de configuraciones estáticas.

Funcionalidades de Eureka Server

Alta Disponibilidad: Eureka Server puede ser configurado en un clúster para alta disponibilidad, replicando el registro de servicios entre múltiples instancias.

Salud y Disponibilidad de Servicios: Eureka Server puede realizar chequeos de salud periódicos a los servicios registrados para asegurarse de que están funcionando correctamente. Los servicios no saludables pueden ser removidos del registro para evitar que otros servicios intenten comunicarse con ellos



Establecer comunicación síncrona entre microservicios utilizando HTTP REST es una práctica común en la arquitectura de microservicios. A continuación, se describe cómo implementar esta comunicación utilizando Spring Boot y las herramientas de Spring Cloud.

Pasos para Establecer Comunicación Síncrona con HTTP REST

- 
1. Configurar Eureka Server (opcional pero recomendado)
 2. Crear Microservicios
 3. Registrar Microservicios con Eureka Server (si se usa)
 4. Implementar Comunicación HTTP REST
 5. Manejo de Fallos y Tiempos de Espera